

INDIVIDUAL ASSIGNMENT

CT101-3-3-IOT

INTERNET OF THINGS: CONCEPTS AND APPLICATION

APD3F2601CS(CYB)

SUBMISSION DATE	**: 26th April 2026**
WEIGHTAGE	**: 60%**
LECTUERE NAME	**:** MR. KAMALANATHAN A/L SHANMUGAM
STUDENT NAME	**:** HEMANTHRAJAH A/L N KRISHNARAJAH
STUDENT TP NUM	**:** TP070844
ASSIGNMENT TITLE	**: Smart Waste Bin Monitoring and Alert System**

Table of Contents

1.0 Introduction 4
1.1 Overview of Arduino 4
1.2 Project Background 5
2.0 Components Required 6
2.1 Maker UNO Board 7
2.2 Breadboard 8.5x5.5cm (400 Holes) 8
2.3 Jumper Wires 9
2.4 I2C 1602 Serial LCD 10
2.5 DHT11 Temperature and Humidity Sensor 10
2.6 HC-SR04 Ultrasonic Sensor 11
2.7 Tilt Switch Sensor 12
2.8 Push Button 13
2.9 Active Buzzer 14
2.10 SG90 Servo Motor 15
2.11 Resistor 16
2.12 Light Emitting Diodes (LED) 17
3.0 System Design 18
3.1 Step 1 - Maker UNO to Breadboard 18
3.2 Step 2 - Connecting I2C 1602 Serial LCD 19
3.3 Step 3 - Connecting DHT11 Sensor 20
3.4 Step 4 - Connecting HC-SR04 Ultrasonic Sensor 21
3.5 Step 5 - Connecting Tilt Switch and Push Button 22
3.6 Step 6 - Connecting Buzzer and LEDs 23
3.7 Step 7 - Connecting SG90 Servo and Completing the System 24
3.8 System Block Diagram 25
3.9 Circuit Diagram 26
3.10 Flowchart / System Logic 27

4.0 Prototype 28
4.1 Final Prototype 28
4.1.1 Lid Open Condition 29
4.1.2 Tilt Alert Condition 30
4.2 Hardware Wiring 31
4.3 System Testing 32
5.0 Source Code 33
5.1 Pre-Operation Source Code 33
5.1.1 Libraries 33
5.1.2 Pin definition and system settings 34
5.1.3 Setup Function 36
5.2 Operation Source Code 37
5.2.1 Main Loop Function 37
5.2.2 Serial Monitor Output 38
5.3 Functions 39
5.3.1 Temperature and Humidity Sensor Functions 39
5.3.2 Ultrasonic Distance Function 40
5.3.3 Tilt and Button Logic Functions 40
5.3.4 Servo and Lid Control Functions 41
5.3.5 LED and Buzzer Functions 42
5.3.6 LCD Update Function 43
5.3.7 Startup Sequence Function 44
6.0 Limitations 45
7.0 Future Enhancement 45
8.0 Conclusion 46
9.0 Prototype Circuit Link 46
10.0 References 47
Appendix A: Full Source Code 48

1.0 Introduction

The Internet of Things (IoT) is a popular technology for enhancing the convenience, safety and efficiency of today's urban services. Sensors, actuators and microcontrollers are used in IoT systems to gather data in real time, analyse the data, and perform an action in response to a change in the environment without human intervention. This is desirable in a smart city because waste disposal, parking, safety, and environmental monitoring services need quick information and responses.

The project is an Arduino-based Smart Waste Bin Monitoring and Alert System to provide hygiene, convenience and safety features in waste disposal. The prototype automatically opens the lid when the user places their hand or object in front of the ultrasonic sensor, shows the environmental data to the user via an LCD display and provides a visual and an audio warning if the waste bin is disturbed or tilted. This project uses automation, sensor, LCD display and alarm to create a small, affordable prototype for the Smart City Exhibition.

The report is organised as per the assignment brief, and is based on the sample report provided, with necessary adjustments for the current smart waste bin project. It details the role of Arduino, the background of the chosen application, the system components and design, the implementation of the prototype, the logic of the source code, limitations and future work. The report also highlights the issues that were encountered in the project: unavailability of a ready-made jumper wire (male-to-female) and the need to modify the jumper wire (male-to-male) to get a temporary physical connection.

1.1 Overview of Arduino

Arduino is an open-source hardware and software platform that uses programmable microcontroller boards and a user-friendly development environment. Arduino is widely used in educational, smart systems, automation, healthcare, industrial monitoring, robotics and home automation projects as it enables users to interface sensors and output devices with code. Arduino boards can detect digital and analog signals, process sensor readings and drive actuators like LEDs, buzzers, displays, relays and motors.

In this project, the Maker UNO is used as the controller. It senses the DHT11 temperature and humidity sensor, HC-SR04 ultrasonic distance sensor, tilt switch, and push button. It then

decides, according to the logic implemented, the I2C LCD, active buzzer, three LEDs and SG90 servo motor. This demonstrates how Arduino enables modern smart systems, being the intermediary between the physical world and the smart reaction.

The student learnt from this project that IoT projects are not just about coding, but also wiring, libraries, power, sensors, hardware test and troubleshooting. All of these things needed to be tested and calibrated, such as LCD address setting, DHT11 pin layout, and the functionality of the tilt switch.

1.2 Project Background

The chosen project is an intelligent waste management prototype as waste disposal is a tangible urban challenge. Traditional waste bins in the home, classroom, office, campus and public spaces are open and often require user to lift the bin lid, which diminishes hygiene. They also lack real-time feedback when being tampered with, moved, or tilted. These make waste bins an ideal candidate for the application of IoT.

The Smart Waste Bin Monitoring and Alert System has three key features. First, it allows for touchless lid opening via HC-SR04 ultrasonic sensor and SG90 servo motor. Second, it measures the ambient temperature and humidity with the DHT11 module and shows it on the LCD. Third, it can sense incorrect usage with a tilt switch and triggers an alarm with LED and buzzer. This enables the system to satisfy the assignment criteria of using at least two sensors and at least three output devices.

This system is fit for a smart city because it enhances convenience, minimises contact with the bin lid, and provides a basic alert system. While the prototype is local and does not have wireless communication yet, it has the basic IoT process of sensing, processing and responding to the environment.

2.0 Components Required

No.	Component	Quantity
1	Maker UNO Board	1
2	840 holes breadboard	1
3	Male-to-male jumper wires	1 set
4	Male-to-female jumper wires	1 set / DIY-adapted where required
5	I2C 1602 Serial LCD, PCF8574T-based	1
6	DHT11 temperature and humidity sensor module	1
7	HC-SR04 ultrasonic sensor	1
8	Tilt switch sensor	1
9	Push button	1
10	Active buzzer	1
11	SG90 servo motor	1
12	Light emitting diode (LED)	3
13	220-ohm resistor	3

Table 1: Components Table

2.1 Maker UNO Board

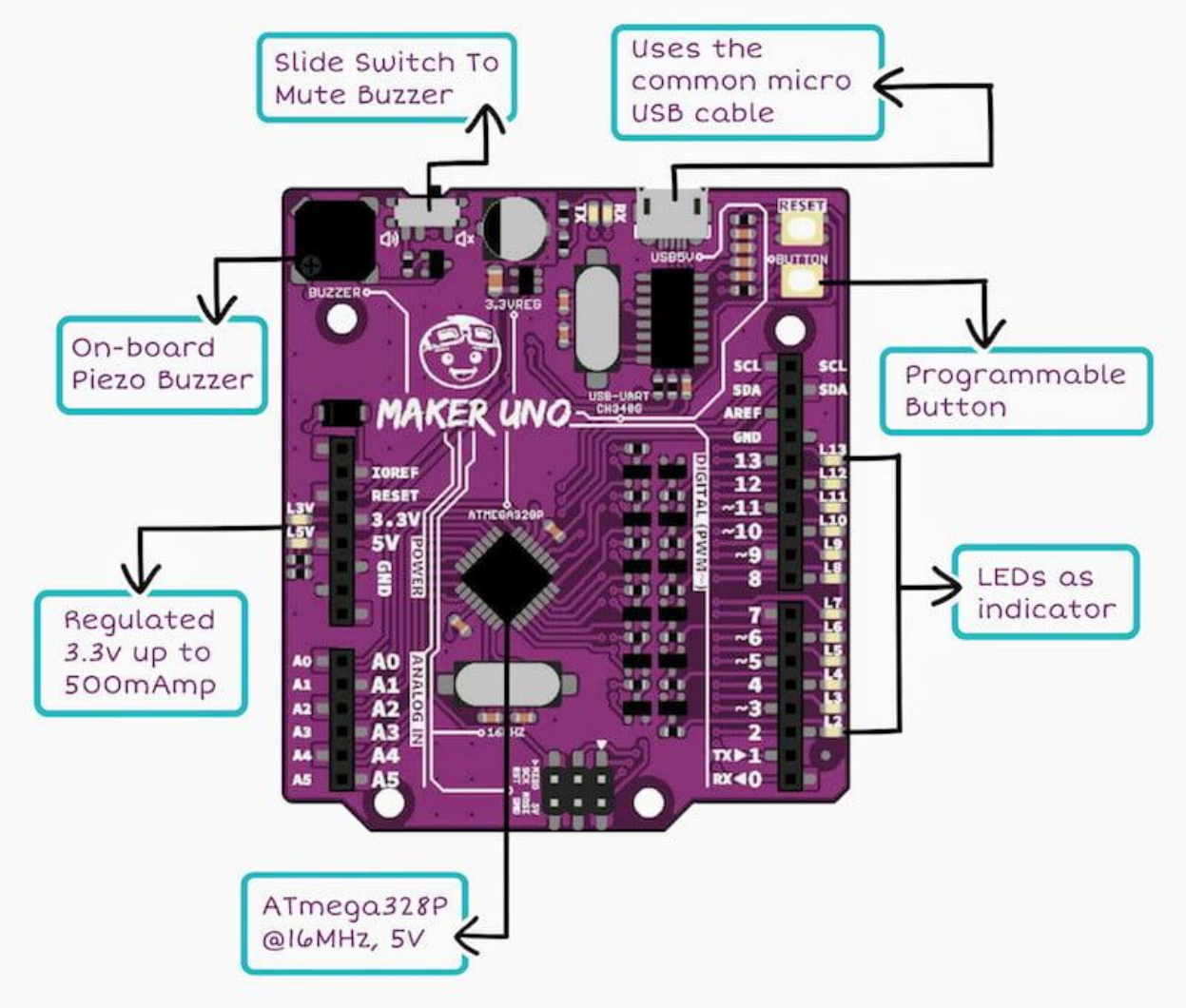


Figure 1: Maker UNO Board

The Maker UNO board is the main controller used in the system. It receives signals from the DHT11 module, HC-SR04 ultrasound sensor, tilt switch and the button. It also drives the LCD, LEDs, buzzer and SG90 servo motor based on the source code. This board was chosen to be compatible with the Arduino IDE and has sufficient digital and analog input and output pins for the prototype.

2.2 Breadboard 8.5x5.5cm (400 Holes)

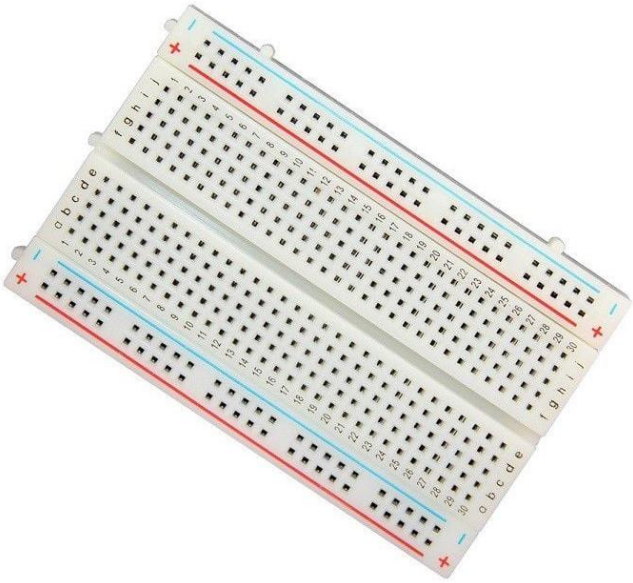


Figure 2: Breadboard 8.5x5.5cm (400 Holes)

The breadboard is used to make the connection without solder. It has power rails for 5V and GND and the sensors and output devices can be easily connected for testing. The breadboard was helpful during the testing stages because the connections could easily be modified to test the LCD, DHT11 and tilt switch.

2.3 Jumper Wires

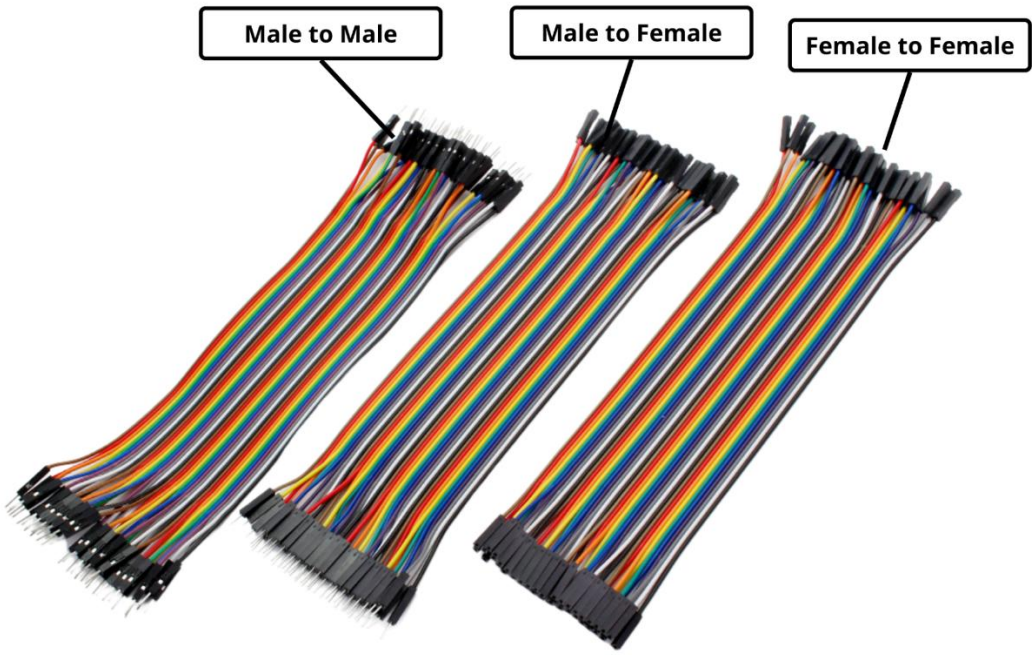


Figure 3: Jumper Wires

Jumper wires are needed to connect the Maker UNO, breadboard, sensors and the outputs. Male-to-male jumper wires were used to connect to the breadboard, while some modules required female jumper wires. In the physical build, a male to female jumper wire was not readily available, so a male to male jumper was carefully hacked to temporarily create a male to female jumper wire to finish the prototype. This was a temporary fix and should be replaced with jumper wires.

2.4 I2C 1602 Serial LCD

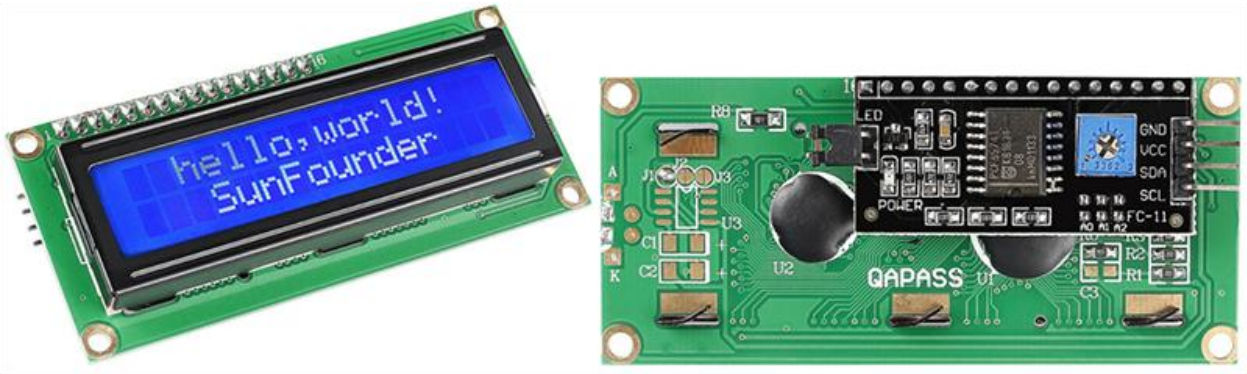


Figure 4: I2C 1602 Serial LCD

The primary display of the project is I2C 1602 Serial LCD. It displays the temperature, humidity, distance, lid position and tilt warning message. It is I2C LCD, so only two pins (SDA and SCL) are required. In this case, SDA is connected to A4 and SCL to A5, making it less cluttered than a normal parallel LCD.

2.5 DHT11 Temperature and Humidity Sensor

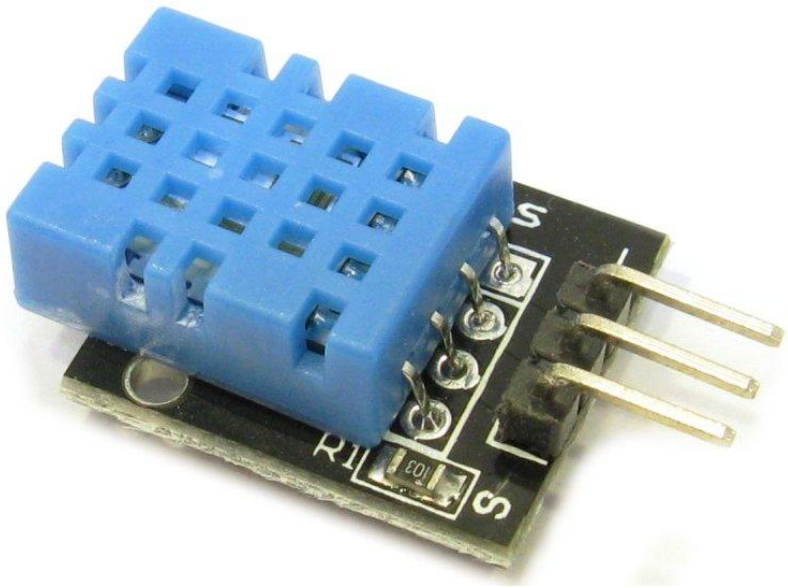


Figure 5: DHT11 Temperature and Humidity Sensor

The DHT11 sensor measures the temperature and humidity. The data pin is connected to digital pin 2. The source code reads the temperature and humidity values, and shows them on the LCD. In case of a sensor reading error, the code shows T:Err or H:Err for debugging.

2.6 HC-SR04 Ultrasonic Sensor



Figure 6: HC-SR04 Ultrasonic Sensor

The HC-SR04 ultrasonic sensor is used to sense whether there is a hand or object in front of the bin. It emits an ultrasonic pulse from the TRIG pin and measures the time taken for the pulse to return via the ECHO pin. The program then calculates the distance in centimetres. It opens the lid if the distance is 15 cm or less.

2.7 Tilt Switch Sensor

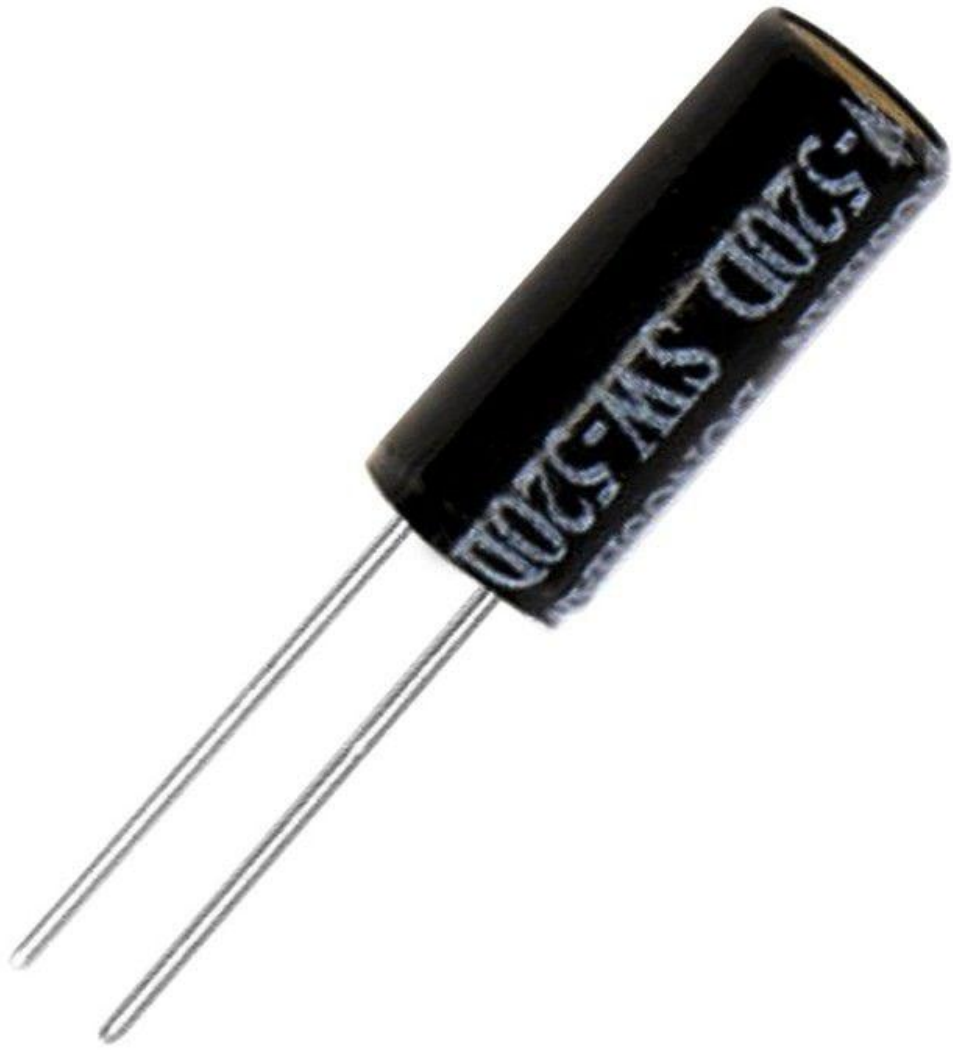


Figure 7: Tilt Switch Sensor

The tilt switch sensor is used to detect the tilting of the bin. In the code, the sensor is considered activated when the input is HIGH, as it was determined that the tilt sensor (chosen for this project) went HIGH when the box was tilted. Once detected, the system goes into alert mode, turns on the alert LED, beeps the buzzer, and prints ALERT: TILT on the LCD.

2.8 Push Button



Figure 8: Push Button

The push button is a reset button. It is wired to digital pin 4 and makes use of the internal pull-up in the Arduino code. While the system is in tilt alert mode, the push button is used to reset the alert system and resume normal operation.

2.9 Active Buzzer



Figure 9: Active Buzzer

The active buzzer makes the sound. It is connected to digital pin 8. The code uses the tone() function to provide a continuous warning beep during tilt alert mode. This can be more effective than just using an LED.

2.10 SG90 Servo Motor

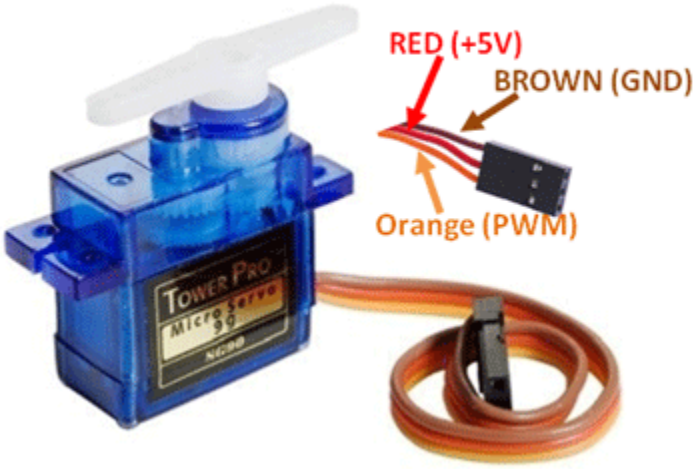


Figure 10: SG90 Servo Motor

The SG90 servo motor is used to open and close the bin lid. The signal wire is connected to digital pin 9. In the code, the servo is set to 0 degrees (closed) and 90 degrees (open). The lid is automatically opened for a few seconds if the ultrasonic sensor is triggered.

2.11 Resistor



Figure 11: 220-ohm Resistor

The three LEDs are protected with three 220-ohm resistors. A series resistor is used to prevent excess current and destroy the LED. Having one resistor per LED also allows the LEDs to be individually turned on and off.

2.12 Light Emitting Diodes (LED)



Figure 12: Light Emitting Diodes (LED)

The system is equipped with three LEDs. The ready LED indicates the waiting mode, the active LED indicates the cover-opening condition and the alert LED indicates the tilt warning mode. The user can have immediate visual information about the system.

3.0 System Design

3.1 Step 1 - Maker UNO to Breadboard

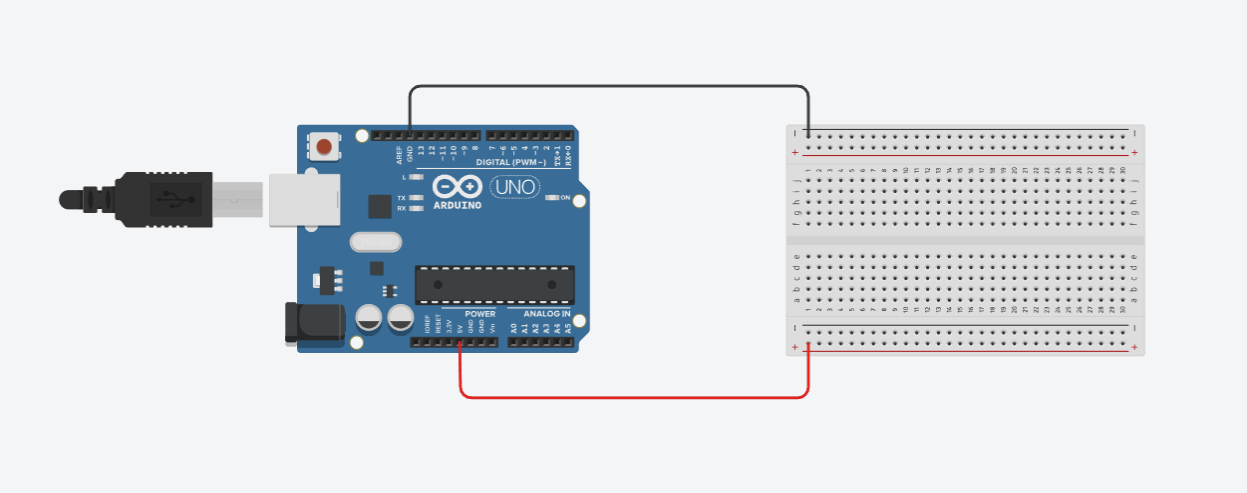


Figure 13: Step 1 system design

The first step is to connect the Maker UNO to the breadboard. The 5V power pin is connected to the positive rail and the GND pin is connected to the negative rail. The rails are connected so that power and ground is available to the other components.

Component / Pin	Connection
Arduino 5V	Breadboard positive rail
Arduino GND	Breadboard ground rail
Top and bottom rails	Bridged for shared 5V and GND

Table 2: Step 1 system design connection summary

3.2 Step 2 - Connecting I2C 1602 Serial LCD

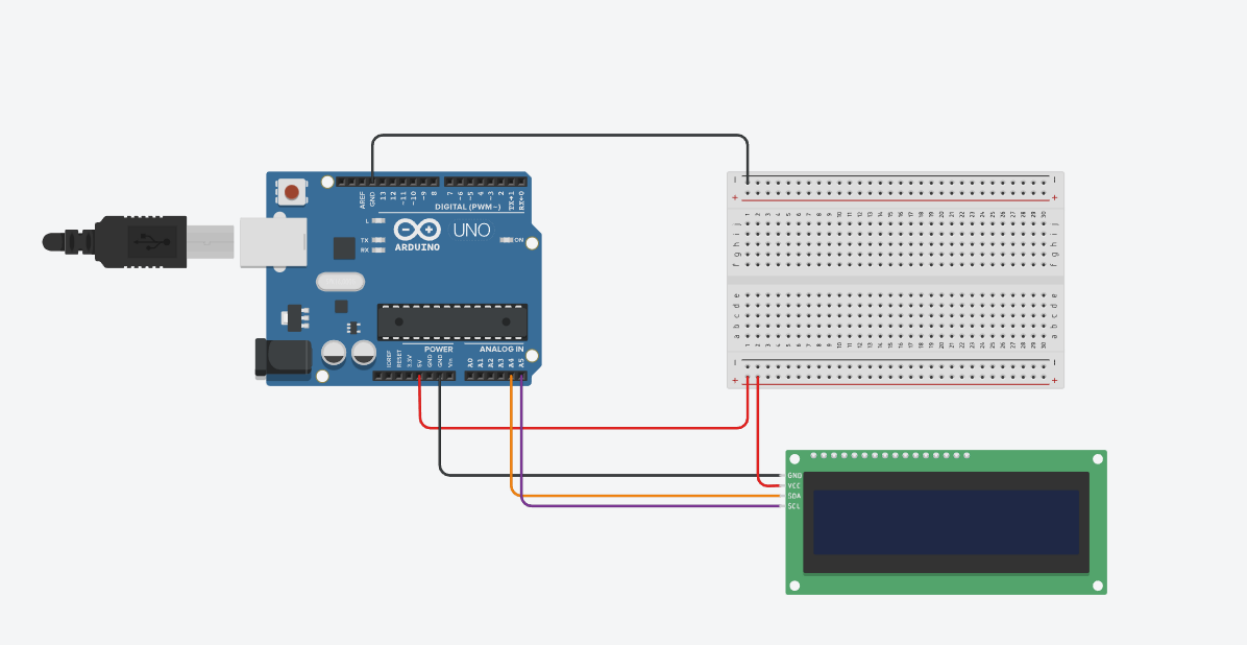


Figure 14: Step 2 system design

The I2C LCD is connected to the Maker UNO using four connections. VCC is connected to 5V, GND is connected to ground, SDA is connected to A4, and SCL is connected to A5. This allows the LCD to display readings and status messages using only two signal lines.

Component / Pin	Connection
LCD GND	GND rail
LCD VCC	5V rail
LCD SDA	Arduino A4
LCD SCL	Arduino A5

Table 3: Step 2 system design connection summary

3.3 Step 3 - Connecting DHT11 Sensor

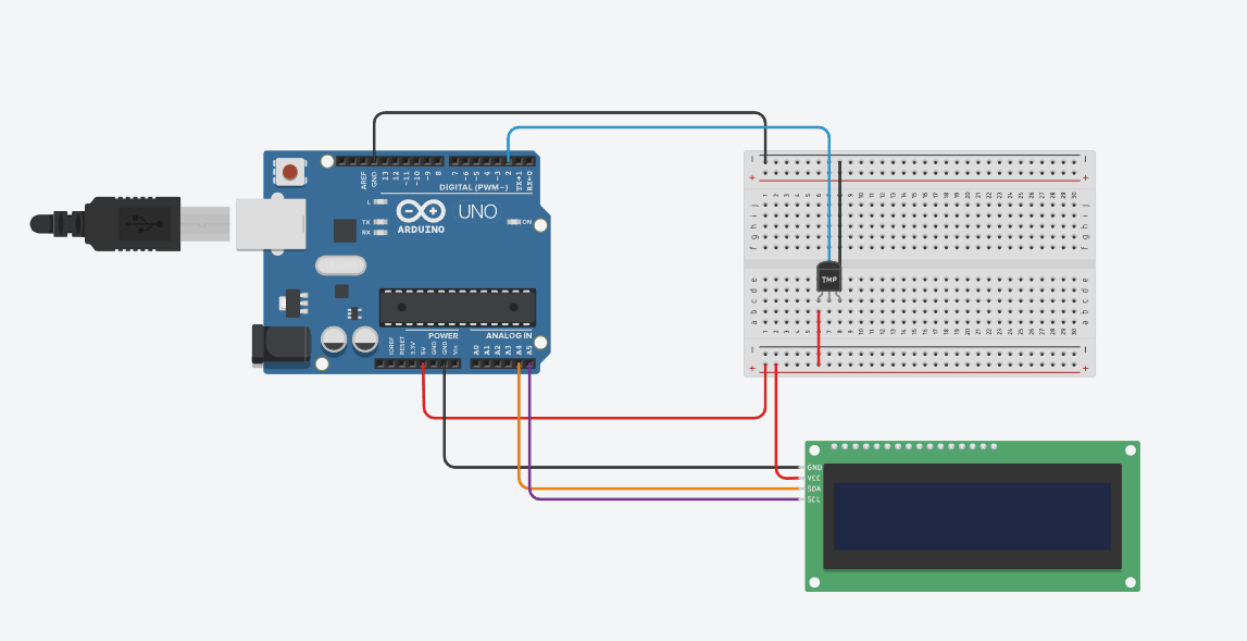


Figure 15: Step 3 system design

The DHT11 module is connected using three pins. The signal pin is connected to D2, VCC is connected to 5V, and GND is connected to ground. This sensor allows the system to display room temperature and humidity values. For this example we will use TMP sensor due to the lack of DHT11 sensor in tinkercad, A0 pin is used as the TMP sensor does not support data pins

Component / Pin	Connection
DHT11 Signal	Arduino D2
DHT11 VCC	5V rail
DHT11 GND	GND rail

Table 4: Step 3 system design connection summary

3.4 Step 4 - Connecting HC-SR04 Ultrasonic Sensor

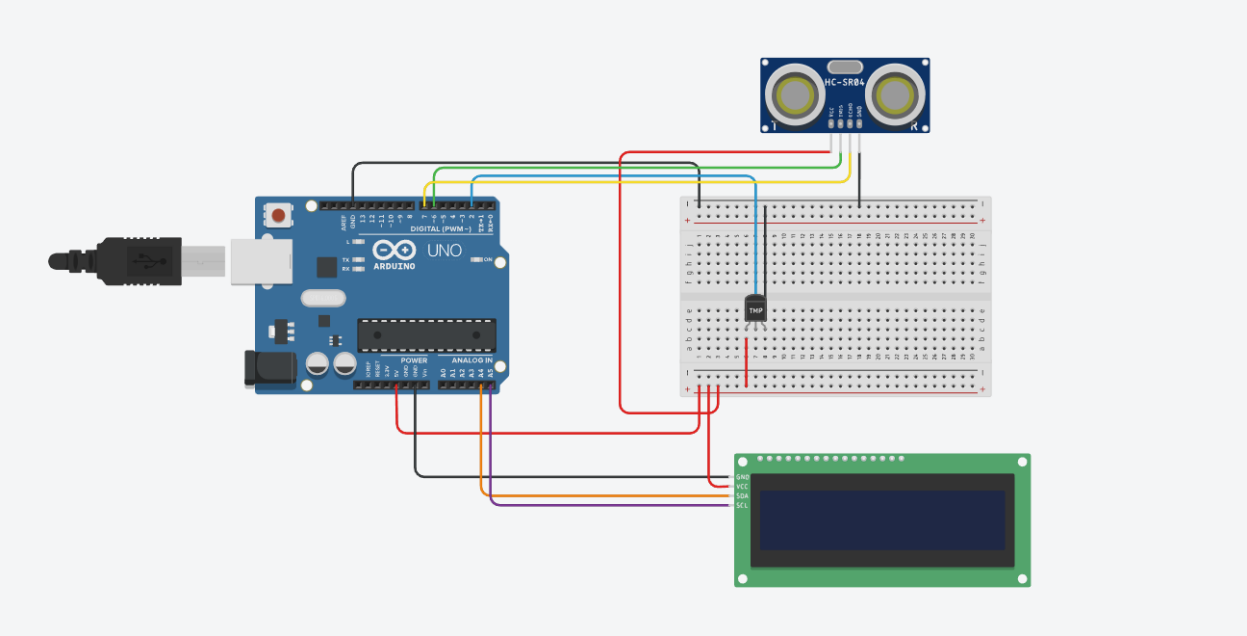


Figure 16: Step 4 system design

The HC-SR04 sensor is connected with VCC to 5V, GND to ground, TRIG to D6, and ECHO to D7. This sensor detects how close an object is to the bin and provides the input required for automatic lid opening.

Component / Pin	Connection
HC-SR04 VCC	5V rail
HC-SR04 GND	GND rail
HC-SR04 TRIG	Arduino D6
HC-SR04 ECHO	Arduino D7

Table 5: Step 4 system design connection summary

3.5 Step 5 - Connecting Tilt Switch and Push Button

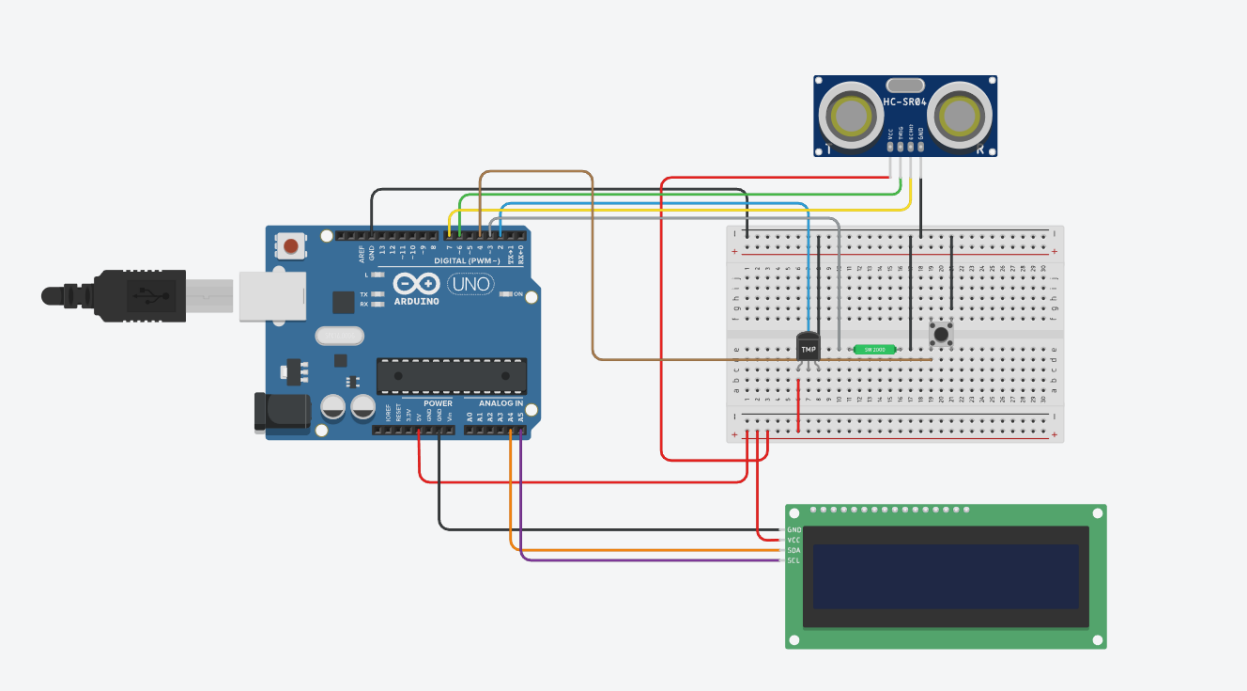


Figure 17: Step 5 system design

The tilt switch is connected between D3 and ground. The push button is connected between D4 and ground. Both inputs use the Arduino internal pull-up setting, so external pull-up or pull-down resistors are not required.

Component / Pin	Connection
Tilt switch	Arduino D3 and GND rail
Push button	Arduino D4 and GND rail
Input mode	INPUT_PULLUP in source code

Table 6: Step 5 system design connection summary

3.6 Step 6 - Connecting Buzzer and LEDs

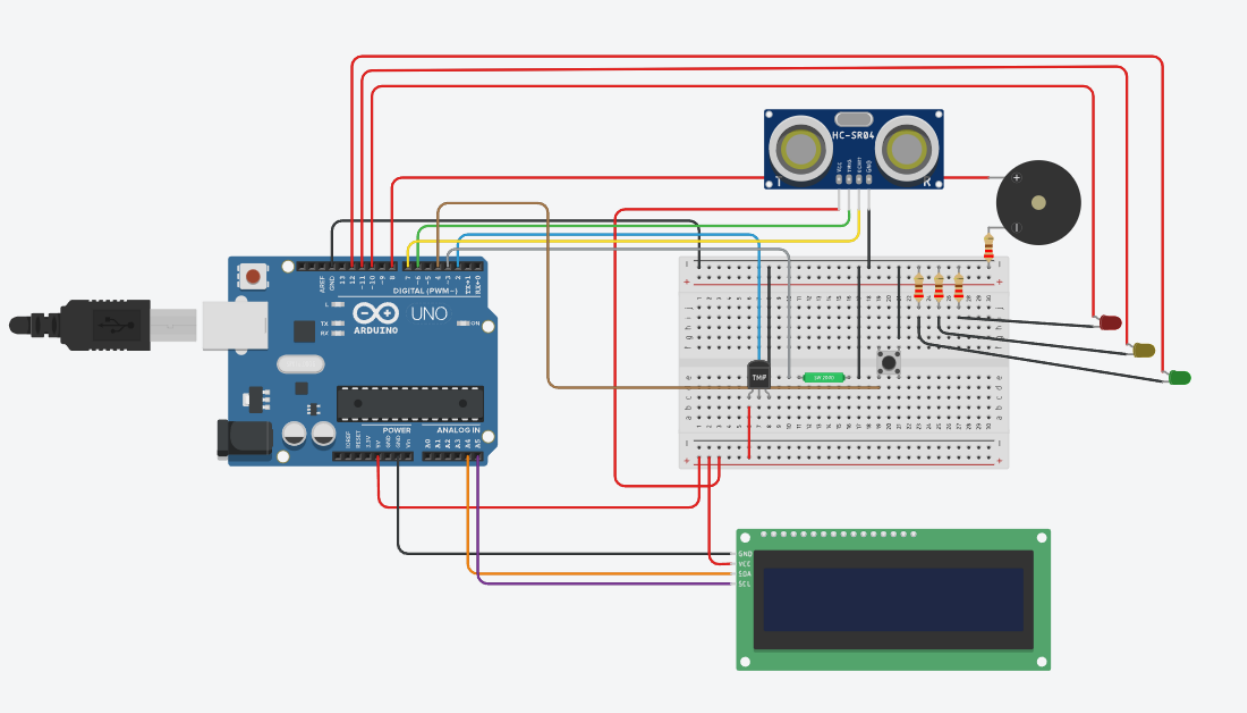


Figure 18: Step 6 system design

The buzzer positive pin is connected to D8 and its negative pin is connected to ground. The ready, active, and alert LEDs are connected to D10, D11, and D12 respectively. Each LED is connected in series with a 220-ohm resistor to ground.

Component / Pin	Connection
Buzzer + / -	D8 / GND rail
Ready LED	D10 through 220-ohm resistor to GND
Active LED	D11 through 220-ohm resistor to GND
Alert LED	D12 through 220-ohm resistor to GND

Table 7: Step 6 system design connection summary

3.7 Step 7 - Connecting SG90 Servo and Completing the System

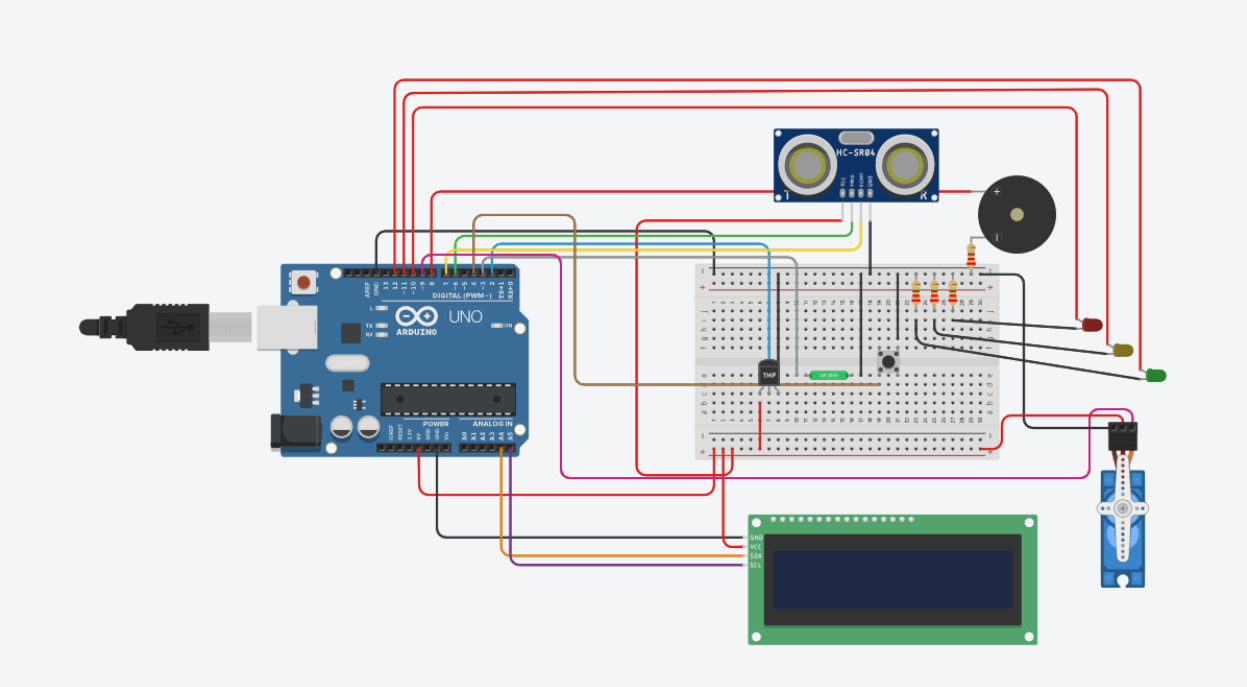


Figure 19: Step 7 system design

The SG90 servo motor is connected with signal to D9, VCC to 5V, and GND to ground. After this step, the complete system can read sensor inputs, display conditions, open the lid, and activate warning outputs.

Component / Pin	Connection
Servo signal	Arduino D9
Servo VCC	5V rail
Servo GND	GND rail

Table 8: Step 7 system design connection summary

3.8 System Block Diagram

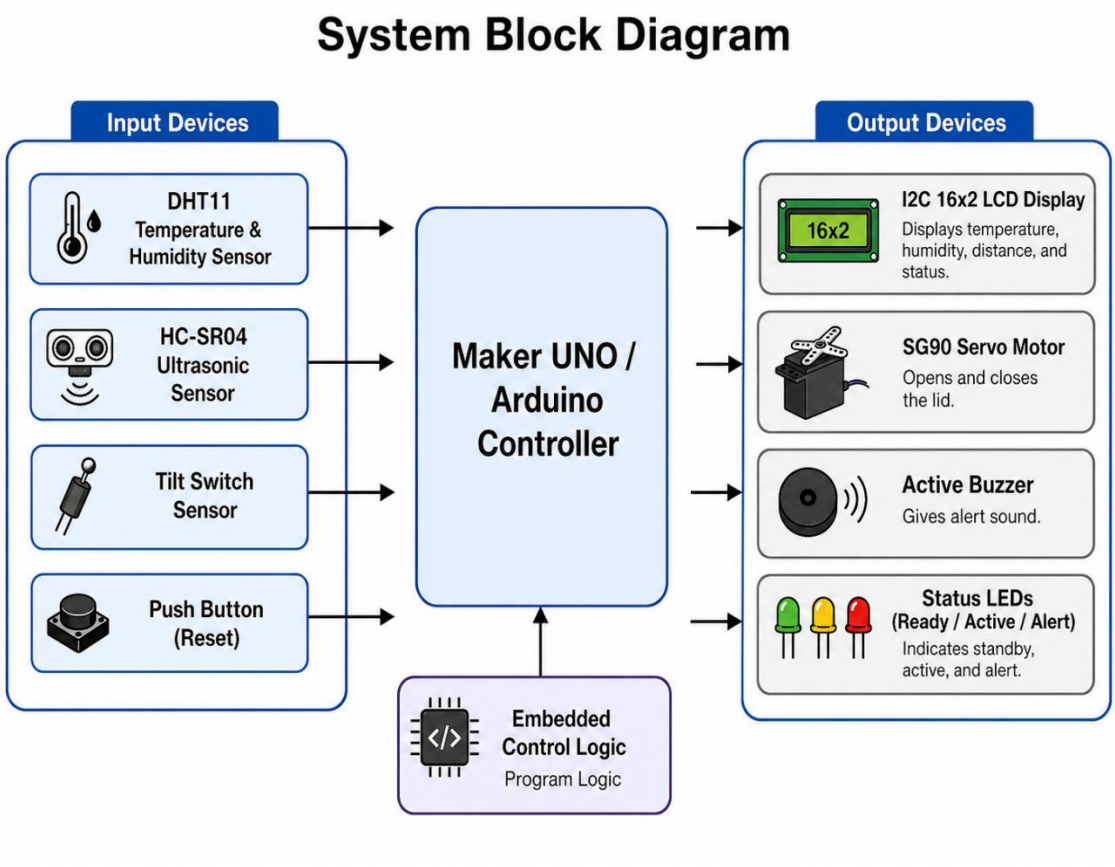


Figure 20: System block diagram

The Maker UNO is shown at the centre of the block diagram. The input section consists of the DHT11 temperature and humidity sensor, HC-SR04 ultrasonic sensor, tilt switch, and push button. The output elements are the I2C LCD, SG90 servo motor, active buzzer, and three status LEDs. This demonstrates the process from sensing through processing and to responding.

3.9 Circuit Diagram

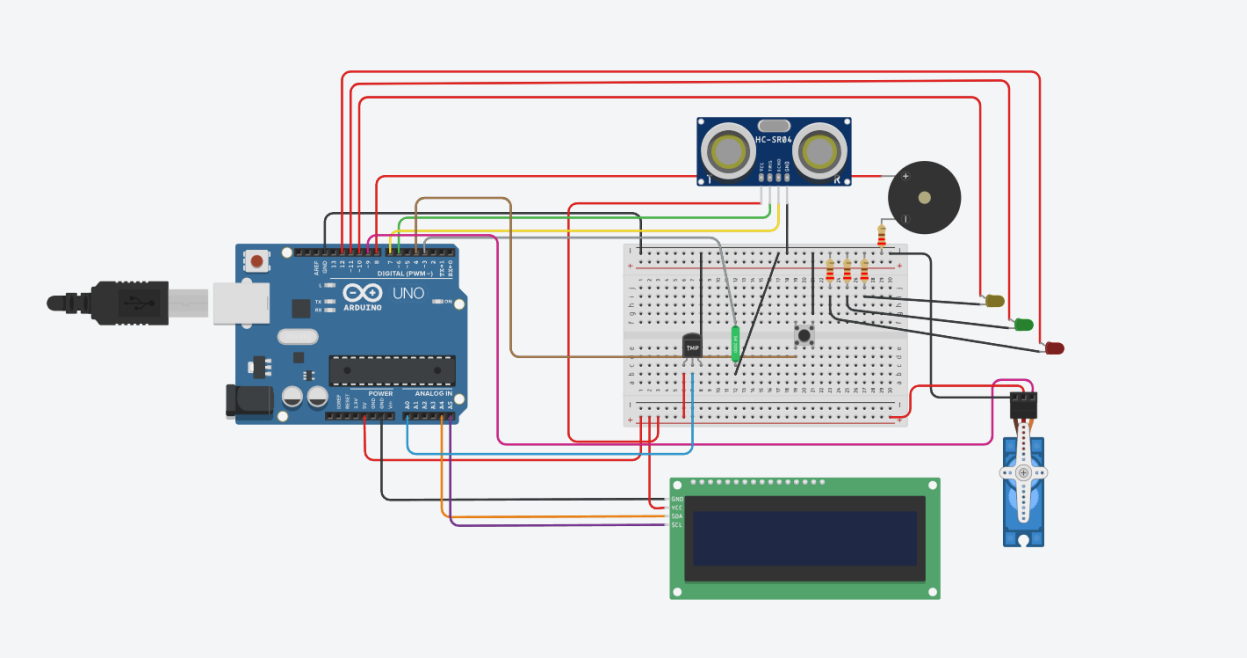


Figure 21: Circuit diagram

The circuit diagram must use the same pin numbers as the source code. D2 is connected to DHT11, D3 to tilt switch, D4 to push button, D6/D7 to the ultrasonic sensor, D8 to the buzzer, D9 to the servo, D10-D12 to the LEDs, and A4/A5 to the I2C LCD. All grounds are tied together into a common ground rail.

3.10 Flowchart / System Logic

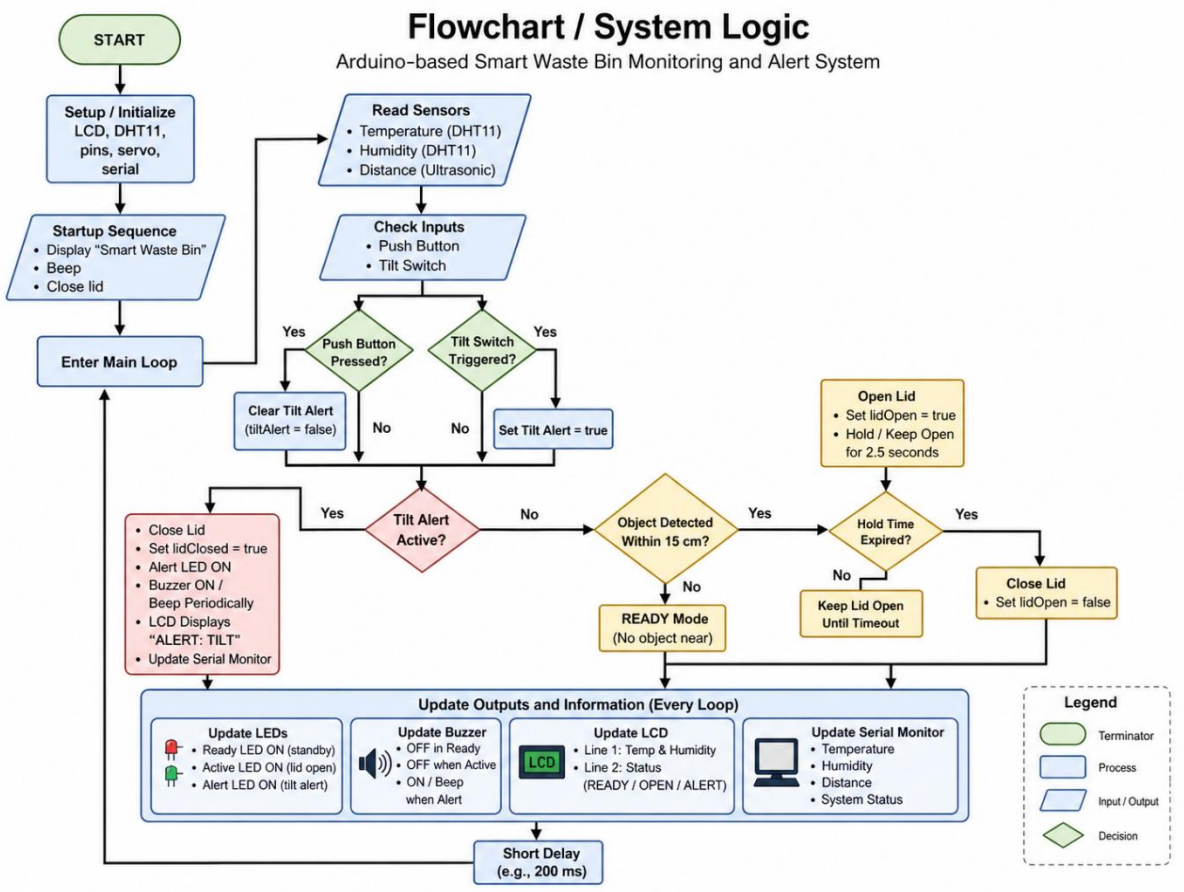


Figure 22: Flowchart / system logic

The flowchart starts with the system initialisation where the LCD, DHT11, ultrasonic sensor, LEDs, buzzer, button, tilt switch and servo are initialised. The main loop reads the temperature, humidity, and distance. Next, it checks for tilt and reset button. If the tilt switch is triggered, the system goes into alert mode, turning on the alert LED, buzzer and displaying a message on the LCD. If there is no tilt alert and there is an object within the specified distance, the servo briefly opens the lid and the active LED is turned on. If neither case is true, the system is in ready mode.

4.0 Prototype

4.1 Final Prototype



Figure 23: Final prototype

The final prototype is housed in a decorated waste-bin. The ultrasonic sensor is at the front to detect a hand or object and the LCD is on the front panel to show temperature, humidity, distance and status. LEDs offer instant feedback and a servo controls the lid. When the system is in its usual state, the LCD shows temperature and humidity in the room with a READY message until the ultrasonic sensor is triggered.

4.1.1 Lid Open Condition



Figure 24: Lid open condition

Once a hand or object is detected by the ultrasonic sensor, the distance measured is less than or equal to the value set in the program. The lid is opened by the servo, the active LED is lit, and the LCD reads LID OPEN. It shows the prototype offers touchless access to the bins.

4.1.2 Tilt Alert Condition



Figure 25: Prototype during tilt alert condition

The system enters the alert condition when the bin is lifted or tilted, causing the tilt switch to change state. This activates the alert LED, the buzzer intermittently sounds an alert, and the LCD message ALERT: TILT is displayed. This feature shows the detection of abnormal handling.

4.2 Hardware Wiring

Component / Pin	Connection	Purpose
LCD GND / VCC / SDA / SCL	GND rail / 5V rail / A4 / A5	LCD display and I2C communication
DHT11 Signal / VCC / GND	D2 / 5V rail / GND rail	Temperature and humidity monitoring
HC-SR04 VCC / GND / TRIG / ECHO	5V rail / GND rail / D6 / D7	Object distance detection
Tilt switch	D3 and GND rail	Tilt or abnormal movement detection
Push button	D4 and GND rail	Manual reset input
Active buzzer + / -	D8 / GND rail	Audio alert output
SG90 Servo Signal / VCC / GND	D9 / 5V rail / GND rail	Automatic lid movement
Ready LED	D10 through 220-ohm resistor to GND	Standby indication
Active LED	D11 through 220-ohm resistor to GND	Lid open indication
Alert LED	D12 through 220-ohm resistor to GND	Tilt alert indication

Table 9: Hardware Wiring Summary

The wiring uses a shared 5V rail and common ground rail. This is important because the LCD, DHT11, ultrasonic sensor, servo, buzzer, and LEDs all require a stable reference ground. During development, the wiring had to be checked several times because an incorrect DHT11 pin arrangement and unavailable female-ended cable caused temporary sensor errors.

4.3 System Testing

No.	Test Case	Input / Condition	Expected Output	Result
1	Standby mode	No object near sensor and no tilt	LCD shows temperature/humidity and READY status; ready LED is ON; lid remains closed	Pass
2	Automatic lid opening	Object detected within 15 cm	Servo opens lid; active LED turns ON; LCD displays LID OPEN	Pass
3	Tilt alert	Bin is tilted or lifted	Alert LED turns ON; buzzer sounds; LCD displays ALERT: TILT	Pass
4	Reset operation	Button is pressed during alert	Alert is cleared; buzzer stops; system returns to ready mode	Pass
5	LCD output	System powered and LCD connected through I2C	Startup message and live status are displayed	Pass

Table 10: System Testing Summary

5.0 Source Code

5.1 Pre-Operation Source Code

5.1.1 Libraries

sketch_IOT_Assignment_Final.ino

```
1   #include <Wire.h>
2   #include <LiquidCrystal_I2C.h>
3   #include <Servo.h>
4   #include <DHT.h>
5
```

Figure 26: Libraries used in the Arduino program

This program uses the libraries Wire, LiquidCrystal_I2C, Servo and DHT. Wire library supports I2C communication, LiquidCrystal_I2C library supports LCD, Servo library supports SG90 servo motor, and DHT library supports the DHT11 temperature and humidity sensor. The libraries save having to write low level code and make the program simpler to read.

5.1.2 Pin definition and system settings

```
// ====================================================
// 1. LCD
// ====================================================
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ====================================================
// 2. DHT11 Temperature and Humidity Sensor
// ====================================================
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// ====================================================
// 3. HC-SR04 Ultrasonic Sensor
// ====================================================
const int TRIG_PIN = 6;
const int ECHO_PIN = 7;

// ====================================================
// 4. Tilt Switch
// ====================================================
const int TILT_PIN = 3;

// ====================================================
// 5. Push Button
// ====================================================
const int BUTTON_PIN = 4;

// ====================================================
// 6. Buzzer
// ====================================================
const int BUZZER_PIN = 8;

// ====================================================
// 7. LEDs
// ====================================================
const int LED_READY  = 10;
const int LED_ACTIVE = 11;
const int LED_ALERT  = 12;

// ====================================================
// 8. Servo
// ====================================================
const int SERVO_PIN = 9;
Servo lidServo;
```

Figure 27: Pin definition section

The program defines the pin for each component. The DHT11 temperature sensor uses D2, the tilt switch uses D3, the reset button uses D4, the ultrasonic sensor uses D6 and D7, the buzzer uses D8, the servo uses D9 and the three LEDs use D10, D11 and D12. The code also sets the open and closed position of the servo, the distance threshold to open and the time to keep the lid open.

```
52  // ================================================
53  // System Settings
54  // ================================================
55  const int SERVO_CLOSED_ANGLE = 180;
56  const int SERVO_OPEN_ANGLE   = 0;
57
58  const float OPEN_DISTANCE_CM = 15.0;
59  const unsigned long LID_HOLD_TIME = 2500;
60
61  bool tiltAlert = false;
62  bool lidOpen = false;
63
64  unsigned long lidCloseDeadline = 0;
65  unsigned long lastBeepTime = 0;
66  unsigned long lastLCDTime = 0;
67  unsigned long lastSerialTime = 0;
68
69  // ================================================
70  // Function Declarations
71  // ================================================
72  float readTemperatureC();
73  float readHumidity();
74  float readDistanceCM();
75
76  bool isTiltTriggered();
77  bool isButtonPressed();
78
79  void openLid();
80  void closeLid();
81
82  void handleTiltAndButton();
83  void handleUltrasonic(float distanceCM);
84
85  void updateLEDs();
86  void updateBuzzer();
87  void updateLCD(float tempC, float hum, float distanceCM);
88  void updateSerial(float tempC, float hum, float distanceCM);
89
90  void startupSequence();
91  void beepOnce(int frequency, int durationMs);
```

Figure 28: system settings section

5.1.3 Setup Function

```
93  // ================================================
94  // Setup
95  // ================================================
96  void setup() {
97    // LCD
98    lcd.init();
99    lcd.backlight();
100
101   // DHT11
102   dht.begin();
103
104   // Ultrasonic
105   pinMode(TRIG_PIN, OUTPUT);
106   pinMode(ECHO_PIN, INPUT);
107
108   // Tilt switch and button
109   pinMode(TILT_PIN, INPUT_PULLUP);
110   pinMode(BUTTON_PIN, INPUT_PULLUP);
111
112   // Outputs
113   pinMode(BUZZER_PIN, OUTPUT);
114   pinMode(LED_READY, OUTPUT);
115   pinMode(LED_ACTIVE, OUTPUT);
116   pinMode(LED_ALERT, OUTPUT);
117
118   // Servo
119   lidServo.attach(SERVO_PIN);
120   closeLid();
121
122   // Serial
123   Serial.begin(9600);
124
125   startupSequence();
126 }
```

Figure 29: Setup function

The setup() function sets up the LCD, initialises the DHT11, configures the pins for the ultrasonic sensor, sets the tilt switch and push button as INPUT_PULLUP, configures the outputs, attaches the servo, initialises communication with the Serial Monitor and performs the start-up sequence. This prepares the system for the main loop.

5.2 Operation Source Code

5.2.1 Main Loop Function

```
128  // ======================================================
129  // Main Loop
130  // ======================================================
131  void loop() {
132    float tempC = readTemperatureC();
133    float hum = readHumidity();
134    float distanceCM = readDistanceCM();
135
136    handleTiltAndButton();
137    handleUltrasonic(distanceCM);
138
139    updateLEDs();
140    updateBuzzer();
141    updateLCD(tempC, hum, distanceCM);
142    updateSerial(tempC, hum, distanceCM);
143
144    delay(100);
145  }
```

Figure 30: Main loop function

The loop() function is the main program loop. It reads the temperature, humidity, and distance sensors, and then calls other functions for tilt detection, ultrasonic lid, LED, buzzer, LCD screen output, and Serial Monitor. The code is modular because all of the operations are in separate named functions.

5.2.2 Serial Monitor Output

```
340  // ======================================================
341  // Serial Monitor Output
342  // ======================================================
343  void updateSerial(float tempC, float hum, float distanceCM) {
344    if (millis() - lastSerialTime < 1000) {
345      return;
346    }
347    lastSerialTime = millis();
348
349    Serial.print("Temp: ");
350    if (tempC == -999.0) Serial.print("ERROR");
351    else Serial.print(tempC);
352
353    Serial.print(" C | Humidity: ");
354    if (hum == -999.0) Serial.print("ERROR");
355    else Serial.print(hum);
356
357    Serial.print(" % | Distance: ");
358    Serial.print(distanceCM);
359    Serial.print(" cm | Tilt Alert: ");
360    Serial.print(tiltAlert ? "YES" : "NO");
361    Serial.print(" | Lid: ");
362    Serial.println(lidOpen ? "OPEN" : "CLOSED");
363  }
364
```

Figure 31: Serial monitor output section

Serial Monitor ×

Message (Enter to send message to 'Arduino UNO' on 'COM3')

```
Temp: 27.4 C | Humidity: 59.8 % | Distance: 36.02 cm | Tilt Alert: NO | Lid: CLOSED
Temp: 27.1 C | Humidity: 60.3 % | Distance: 37.10 cm | Tilt Alert: NO | Lid: CLOSED
Temp: 27.6 C | Humidity: 58.9 % | Distance: 40.42 cm | Tilt Alert: NO | Lid: CLOSED
Temp: 27.3 C | Humidity: 61.2 % | Distance: 41.54 cm | Tilt Alert: NO | Lid: CLOSED
Temp: 27.0 C | Humidity: 59.6 % | Distance: 53.78 cm | Tilt Alert: NO | Lid: CLOSED
Temp: 27.5 C | Humidity: 60.1 % | Distance: 3.94 cm | Tilt Alert: NO | Lid: OPEN
Temp: 27.2 C | Humidity: 58.7 % | Distance: 4.60 cm | Tilt Alert: NO | Lid: OPEN
Temp: 27.4 C | Humidity: 60.4 % | Distance: 6.21 cm | Tilt Alert: NO | Lid: OPEN
Temp: 27.1 C | Humidity: 59.3 % | Distance: 42.89 cm | Tilt Alert: NO | Lid: OPEN
Temp: 27.6 C | Humidity: 61.0 % | Distance: 55.31 cm | Tilt Alert: NO | Lid: OPEN
Temp: 27.3 C | Humidity: 59.9 % | Distance: 42.48 cm | Tilt Alert: NO | Lid: CLOSED
```

Figure 32: Serial monitor output section

The updateSerial() function outputs temperature, humidity, distance, tilt alarm status and lid status to the Serial Monitor each second. This was useful when testing the code, as the student could check whether the sensors were reporting reasonable values and check the logic was changing as expected.

5.3 Functions

5.3.1 Temperature and Humidity Sensor Functions

```
147  // ======================================================
148  // 2. DHT11 Functions
149  // ======================================================
150  float readTemperatureC() {
151    float tempC = dht.readTemperature();
152
153    if (isnan(tempC)) {
154      return -999.0;
155    }
156
157    return tempC;
158  }
159
160  float readHumidity() {
161    float hum = dht.readHumidity();
162
163    if (isnan(hum)) {
164      return -999.0;
165    }
166
167    return hum;
168  }
```

Figure 33: DHT11 sensor functions

The readTemperatureC() and readHumidity() functions read the DHT11 sensor. If the sensor does not return a value, it returns -999.0. The LCD function then displays T:Err or H:Err, which stops the program from crashing and lets the user know if there is an issue with the sensor.

5.3.2 Ultrasonic Distance Function

```
170  // ======================================================
171  // 3. HC-SR04 Function
172  // ======================================================
173  float readDistanceCM() {
174    digitalWrite(TRIG_PIN, LOW);
175    delayMicroseconds(2);
176
177    digitalWrite(TRIG_PIN, HIGH);
178    delayMicroseconds(10);
179    digitalWrite(TRIG_PIN, LOW);
180
181    unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);
182
183    if (duration == 0) {
184      return -1.0;
185    }
186
187    return duration * 0.0343 / 2.0;
188  }
189
```

Figure 34: Ultrasonic distance function

The readDistanceCM() function triggers a pulse in TRIG pin and measures the time in ECHO pin. This time is then converted to centimetres using the speed of sound. The distance is used to determine whether to open the lid using the servo.

5.3.3 Tilt and Button Logic Functions

```
190  // ======================================================
191  // 4. Tilt Switch Function
192  // INPUT_PULLUP is still used.
193  // Adjusted for your sensor behavior:
194  // HIGH = triggered
195  // LOW  = normal
196  // ======================================================
197  bool isTiltTriggered() {
198    return digitalRead(TILT_PIN) == HIGH;
199  }
200
```

Figure 35: Tilt switch and push button functions

The isTiltTriggered() function checks the tilt switch. In the final version, HIGH means triggered because this tilt sensor returned HIGH when tilted. The isButtonPressed() function reads the reset button using INPUT_PULLUP, so LOW is button pressed.

5.3.4 Servo and Lid Control Functions

```
211  // ======================================================
212  // 8. Servo Functions
213  // ======================================================
214  void openLid() {
215    lidServo.write(SERVO_OPEN_ANGLE);
216  }
217
218  void closeLid() {
219    lidServo.write(SERVO_CLOSED_ANGLE);
220  }
221
222  // ======================================================
223  // Tilt and Button Logic
224  // ======================================================
225  void handleTiltAndButton() {
226    if (isButtonPressed()) {
227      tiltAlert = false;
228    }
229
230    if (isTiltTriggered()) {
231      tiltAlert = true;
232    }
233  }
234
```

Figure 36: Servo open and close functions

The openLid() and closeLid() functions are used to control the SG90 servo motor. The lid is open (90 degrees) when the object is in range and closed (0 degrees) after the hold time expires or when tilt alert triggers.

5.3.5 LED and Buzzer Functions

```
// ====================================================
// 7. LED Logic
// READY  = normal standby
// ACTIVE = lid open / object detected
// ALERT  = tilt warning
// ====================================================
void updateLEDs() {
  if (tiltAlert) {
    digitalWrite(LED_READY, LOW);
    digitalWrite(LED_ACTIVE, LOW);
    digitalWrite(LED_ALERT, HIGH);
  } else if (lidOpen) {
    digitalWrite(LED_READY, LOW);
    digitalWrite(LED_ACTIVE, HIGH);
    digitalWrite(LED_ALERT, LOW);
  } else {
    digitalWrite(LED_READY, HIGH);
    digitalWrite(LED_ACTIVE, LOW);
    digitalWrite(LED_ALERT, LOW);
  }
}

// ====================================================
// 6. Buzzer Logic
// ====================================================
void updateBuzzer() {
  if (tiltAlert) {
    if (millis() - lastBeepTime >= 500) {
      tone(BUZZER_PIN, 1200, 150);
      lastBeepTime = millis();
    }
  } else {
    noTone(BUZZER_PIN);
  }
}

```

Figure 37: LED and Buzzer Functions

The updateLEDs() function turns on/off the ready, active and alert LEDs based on system conditions. The updateBuzzer() function plays a repeating warning tone when tiltAlert is true and turns the tone off when the alert is off.

5.3.6 LCD Update Function

```
// ================================================
// 1. LCD Update
// ================================================
void updateLCD(float tempC, float hum, float distanceCM) {
  if (millis() - lastLCDTime < 300) {
    return;
  }
  lastLCDTime = millis();

  // Line 1
  lcd.setCursor(0, 0);

  if (tempC == -999.0) {
    lcd.print("T:Err ");
  } else {
    lcd.print("T:");
    lcd.print(tempC, 1);
    lcd.print("C ");
  }

  if (hum == -999.0) {
    lcd.print("H:Err ");
  } else {
    lcd.print("H:");
    lcd.print((int)hum);
    lcd.print("% ");
  }

  // Clear tail
  lcd.print("   ");

  // Line 2
  lcd.setCursor(0, 1);

  if (tiltAlert) {
    lcd.print("ALERT: TILT    ");
  } else if (lidOpen) {
    lcd.print("LID OPEN       ");
  } else if (distanceCM > 0) {
    lcd.print("READY ");
    lcd.print((int)distanceCM);
    lcd.print("cm     ");
  } else {
    lcd.print("READY          ");
  }
}
```

Figure 38: LCD Update Function

The updateLCD() function shows the user interface. The top row displays the temperature and relative humidity. The second row displays READY, LID OPEN or ALERT: TILT, based on the current state. So, the LCD is the major user-interface device of the prototype.

5.3.7 Startup Sequence Function

```
365  // ======================================================
366  // Startup Sequence
367  // ======================================================
368  void startupSequence() {
369    lcd.clear();
370    lcd.setCursor(0, 0);
371    lcd.print("Smart Waste Bin");
372    lcd.setCursor(0, 1);
373    lcd.print("Starting...");
374
375    digitalWrite(LED_READY, LOW);
376    digitalWrite(LED_ACTIVE, LOW);
377    digitalWrite(LED_ALERT, LOW);
378
379    closeLid();
380
381    beepOnce(1000, 120);
382    delay(150);
383    beepOnce(1200, 120);
384    delay(150);
385    beepOnce(1400, 120);
386    delay(500);
387
388    lcd.clear();
389  }
390
391  void beepOnce(int frequency, int durationMs) {
392    tone(BUZZER_PIN, frequency, durationMs);
393    delay(durationMs);
394    noTone(BUZZER_PIN);
395  }
```

Figure 39: Startup Sequence Function

The startupSequence() function prints Smart Waste Bin and Starting on the LCD, turns all the LEDs off, closes the lid and beeps the buzzer a few times. This provides a clear indication of the system starting up.

6.0 Limitations

The first is the DHT11 module's accuracy. DHT11 is a basic temperature and humidity sensor, only suitable for educational and demonstration purposes, with less accuracy and slower response than more sophisticated sensors. So the LCD readings of room temperature and humidity are only approximate, and not suitable for scientific research.

The second constraint is that it is a local system. The system is not yet connected to the internet, Bluetooth, or the cloud, so the user can only read information from the LCD and sensors. A smart-city system would need remote data monitoring and storage to better analyse waste management.

The third issue is related to prototyping. Wires can fall out of the breadboard, particularly when the prototype is tilted or shaken. During the development, the lack of a male-to-female jumper wire meant that a male to male wire had to be modified. While this enabled the prototype to be built, it is less tidy and stable than using the right wire.

The fourth drawback is that the servo motor is connected to the same 5V power supply as the logic. This is fine for the small prototype, but in a more complex design, it would be better to have a regulated supply for the motor. Also, the current prototype is able to detect user presence and tilt, but it does not detect the level of waste in the bin.

7.0 Future Enhancement

Future enhancements could include wireless communication and monitoring. A Wi-Fi or Bluetooth module could be added, allowing the smart waste bin to send tilt and fill-level notifications to a mobile phone or PC. A cloud-based system could also be used to monitor multiple bins from a single or centralized platform, making the project more interesting to smart-cities.

The sensing system could also be improved. An additional ultrasonic sensor could be installed within the bin to determine fill level, and a DHT22 sensor (or a better environmental sensor) could be used to measure temperature and humidity, instead of the DHT11. A moisture sensor could also be incorporated if an extension to the system is made for sensing wet and dry waste.

The system could also be improved by either soldering the connections or using a printed circuit board instead of wiring on the breadboard. The jumper wire made from a DIY wire should also be replaced with male-to-female jumper wires. This would make it cleaner, safer, and more suitable for demonstrations or deployment.

8.0 Conclusion

The Smart Waste Bin Monitoring and Alert System shows how IoT elements can be integrated to build an IoT prototype of a smart city. The system employs the DHT11 module, HC-SR04 sensor, tilt switch, push button, LCD, LEDs, buzzer and servo motor to offer environmental sensing, lid opening without physical contact, and tilt warning system. It meets the assignment criteria of using two or more sensors and three or more output devices.

The assignment also gave hands-on experience in wiring, programming, debugging, installing libraries, setting up LCD, testing sensors and building prototypes. The prototype demonstrates the potential of low-cost components to provide a useful interactive IoT system. While the system is simplistic and can be improved with connectivity, fill level detection, enhanced sensors and hardware design in future iterations.

9.0 Prototype Circuit Link

https://www.tinkercad.com/things/hKktfONldhd/editel?returnTo=%2Fdashboard&sharecode=GQAgVIl9vTp3GP-NKHjO5FGJUB-N1LbjmhzJ4zwzJRg

10.0 References

Adafruit. (2024). Using a tilt sensor. https://learn.adafruit.com/tilt-sensor/using-a-tilt-sensor

Adafruit. (n.d.). DHT11, DHT22 and AM2302 sensors. https://learn.adafruit.com/dht

Arduino. (n.d.). Digital pins. https://docs.arduino.cc/learn/microcontrollers/digital-pins/

Arduino. (n.d.). Servo motor basics with Arduino. https://docs.arduino.cc/learn/electronics/servo-motors/

Arduino. (n.d.). tone(). https://docs.arduino.cc/language-reference/en/functions/advanced-io/tone/

Arus Academy. (n.d.). Maker Uno. https://arusacademy.org.my/web/maker-uno/

BYJU'S. (2024). Light emitting diode (LED). https://byjus.com/physics/light-emitting-diode/

ElectronicWings. (n.d.). DHT11 sensor guide with pinout, working, and Arduino programming. https://www.electronicwings.com/sensors-modules/dht11

Last Minute Engineers. (n.d.). In-depth: Interfacing an I2C LCD with Arduino. https://lastminuteengineers.com/i2c-lcd-arduino-tutorial/

SparkFun. (n.d.). How to use a breadboard. https://learn.sparkfun.com/tutorials/how-to-use-a-breadboard/all

SparkFun. (n.d.). Ultrasonic ranging module HC-SR04 datasheet. https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

TechTarget. (2021, December 3). Resistor. https://www.techtarget.com/whatis/definition/resistor

Appendix A: Full Source Code

The following source code is the final code for the Smart Waste Bin Monitoring and Alert System. It is divided by module and purpose for easy understanding and editing.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
#include <DHT.h>

// ================================================
// 1. LCD
// ================================================
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ================================================
// 2. DHT11 Temperature and Humidity Sensor
// ================================================
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// ================================================
// 3. HC-SR04 Ultrasonic Sensor
// ================================================
const int TRIG_PIN = 6;
const int ECHO_PIN = 7;

// ================================================
// 4. Tilt Switch
// ================================================
const int TILT_PIN = 3;

// ================================================
// 5. Push Button
// ================================================
const int BUTTON_PIN = 4;

// ================================================
// 6. Buzzer
// ================================================
const int BUZZER_PIN = 8;

// ================================================
```

```
// 7. LEDs
// ================================================
const int LED_READY  = 10;
const int LED_ACTIVE = 11;
const int LED_ALERT  = 12;


// ================================================
// 8. Servo
// ================================================
const int SERVO_PIN = 9;
Servo lidServo;

// ================================================
// System Settings
// ================================================
const int SERVO_CLOSED_ANGLE = 0;
const int SERVO_OPEN_ANGLE   = 90;

const float OPEN_DISTANCE_CM = 15.0;
const unsigned long LID_HOLD_TIME = 2500;

bool tiltAlert = false;
bool lidOpen = false;

unsigned long lidCloseDeadline = 0;
unsigned long lastBeepTime = 0;
unsigned long lastLCDTime = 0;
unsigned long lastSerialTime = 0;

// ================================================
// Function Declarations
// ================================================
float readTemperatureC();
float readHumidity();
float readDistanceCM();

bool isTiltTriggered();
bool isButtonPressed();

void openLid();
void closeLid();

void handleTiltAndButton();
```

```
void handleUltrasonic(float distanceCM);

void updateLEDs();
void updateBuzzer();
void updateLCD(float tempC, float hum, float distanceCM);
void updateSerial(float tempC, float hum, float distanceCM);

void startupSequence();
void beepOnce(int frequency, int durationMs);

// ================================================
// Setup
// ================================================
void setup() {
  // LCD
  lcd.init();
  lcd.backlight();

  // DHT11
  dht.begin();

  // Ultrasonic
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  // Tilt switch and button
  pinMode(TILT_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  // Outputs
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_READY, OUTPUT);
  pinMode(LED_ACTIVE, OUTPUT);
  pinMode(LED_ALERT, OUTPUT);

  // Servo
  lidServo.attach(SERVO_PIN);
  closeLid();

  // Serial
  Serial.begin(9600);

  startupSequence();
}
```

```
// ================================================
// Main Loop
// ================================================
void loop() {
  float tempC = readTemperatureC();
  float hum = readHumidity();
  float distanceCM = readDistanceCM();


  handleTiltAndButton();
  handleUltrasonic(distanceCM);

  updateLEDs();
  updateBuzzer();
  updateLCD(tempC, hum, distanceCM);
  updateSerial(tempC, hum, distanceCM);

  delay(100);
}

// ================================================
// 2. DHT11 Functions
// ================================================
float readTemperatureC() {
  float tempC = dht.readTemperature();

  if (isnan(tempC)) {
    return -999.0;
  }

  return tempC;
}

float readHumidity() {
  float hum = dht.readHumidity();

  if (isnan(hum)) {
    return -999.0;
  }

  return hum;
}
```

```
// ================================================
// 3. HC-SR04 Function
// ================================================
float readDistanceCM() {
 digitalWrite(TRIG_PIN, LOW);
 delayMicroseconds(2);

 digitalWrite(TRIG_PIN, HIGH);
 delayMicroseconds(10);
 digitalWrite(TRIG_PIN, LOW);


 unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);

 if (duration == 0) {
  return -1.0;
 }

 return duration * 0.0343 / 2.0;
}

// ================================================
// 4. Tilt Switch Function
// INPUT_PULLUP means:
// LOW  = triggered
// HIGH = normal
// ================================================
bool isTiltTriggered() {
 return digitalRead(TILT_PIN) == LOW;
}

// ================================================
// 5. Push Button Function
// INPUT_PULLUP means:
// LOW  = pressed
// HIGH = released
// ================================================
bool isButtonPressed() {
 return digitalRead(BUTTON_PIN) == LOW;
}

// ================================================
// 8. Servo Functions
// ================================================
```

```
void openLid() {
  lidServo.write(SERVO_OPEN_ANGLE);
}

void closeLid() {
  lidServo.write(SERVO_CLOSED_ANGLE);
}

// ================================================
// Tilt and Button Logic
// ================================================
void handleTiltAndButton() {
  if (isButtonPressed()) {
    tiltAlert = false;
  }

  if (isTiltTriggered()) {
    tiltAlert = true;
  }
}

// ================================================
// Ultrasonic / Lid Logic
// ================================================
void handleUltrasonic(float distanceCM) {
  if (tiltAlert) {
    closeLid();
    lidOpen = false;
    return;
  }

  if (distanceCM > 0 && distanceCM <= OPEN_DISTANCE_CM) {
    openLid();
    lidOpen = true;
    lidCloseDeadline = millis() + LID_HOLD_TIME;
  }

  if (lidOpen && millis() > lidCloseDeadline) {
    closeLid();
    lidOpen = false;
  }
}

// ================================================
```

```
// 7. LED Logic
// READY  = normal standby
// ACTIVE = lid open / object detected
// ALERT  = tilt warning
// ================================================
void updateLEDs() {
 if (tiltAlert) {
   digitalWrite(LED_READY, LOW);
   digitalWrite(LED_ACTIVE, LOW);
   digitalWrite(LED_ALERT, HIGH);
 } else if (lidOpen) {
   digitalWrite(LED_READY, LOW);
   digitalWrite(LED_ACTIVE, HIGH);
   digitalWrite(LED_ALERT, LOW);

 } else {
   digitalWrite(LED_READY, HIGH);
   digitalWrite(LED_ACTIVE, LOW);
   digitalWrite(LED_ALERT, LOW);
 }
}

// ================================================
// 6. Buzzer Logic
// ================================================
void updateBuzzer() {
 if (tiltAlert) {
   if (millis() - lastBeepTime >= 500) {
     tone(BUZZER_PIN, 1200, 150);
     lastBeepTime = millis();
   }
 } else {
   noTone(BUZZER_PIN);
 }
}

// ================================================
// 1. LCD Update
// ================================================
void updateLCD(float tempC, float hum, float distanceCM) {
 if (millis() - lastLCDTime < 300) {
   return;
 }
 lastLCDTime = millis();
```

```
 // Line 1
 lcd.setCursor(0, 0);

 if (tempC == -999.0) {
  lcd.print("T:Err ");
 } else {
  lcd.print("T:");
  lcd.print(tempC, 1);
  lcd.print("C ");
 }

 if (hum == -999.0) {
  lcd.print("H:Err ");
 } else {
  lcd.print("H:");
  lcd.print((int)hum);
  lcd.print("% ");
 }

 // Clear tail
 lcd.print("   ");

 // Line 2
 lcd.setCursor(0, 1);

 if (tiltAlert) {
  lcd.print("ALERT: TILT    ");
 } else if (lidOpen) {
  lcd.print("LID OPEN       ");
 } else if (distanceCM > 0) {
  lcd.print("READY ");
  lcd.print((int)distanceCM);
  lcd.print("cm     ");
 } else {
  lcd.print("READY          ");
 }
}

// ================================================
// Serial Monitor Output
// ================================================
void updateSerial(float tempC, float hum, float distanceCM) {
 if (millis() - lastSerialTime < 1000) {
  return;
```

```
  }
  lastSerialTime = millis();

  Serial.print("Temp: ");
  if (tempC == -999.0) Serial.print("ERROR");
  else Serial.print(tempC);

  Serial.print(" C | Humidity: ");
  if (hum == -999.0) Serial.print("ERROR");
  else Serial.print(hum);

  Serial.print(" % | Distance: ");
  Serial.print(distanceCM);
  Serial.print(" cm | Tilt Alert: ");
  Serial.print(tiltAlert ? "YES" : "NO");
  Serial.print(" | Lid: ");
  Serial.println(lidOpen ? "OPEN" : "CLOSED");
}

// ================================================
// Startup Sequence
// ================================================
void startupSequence() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Smart Waste Bin");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  digitalWrite(LED_READY, LOW);
  digitalWrite(LED_ACTIVE, LOW);
  digitalWrite(LED_ALERT, LOW);

  closeLid();

  beepOnce(1000, 120);
  delay(150);
  beepOnce(1200, 120);
  delay(150);
  beepOnce(1400, 120);
  delay(500);

  lcd.clear();
}
```

```
void beepOnce(int frequency, int durationMs) {
  tone(BUZZER_PIN, frequency, durationMs);
  delay(durationMs);
  noTone(BUZZER_PIN);
}
```